

WSC 주행효율 예측 MPC 시뮬레이터

총괄 기획안

Bridgestone World Solar Challenge 2027 출전 대비
태양광 자동차 구간별 최적 주행속도 예측 시스템

프로젝트명	WSC_DriveEff_Project
저장소	github.com/Dorolong/WSC_Project (Public)
배포 앱 버전	v1.0.8 (Streamlit Cloud)
문서 기준일	2026-08-05
대상 경로	Darwin → Adelaide, 3,038.3 km
원작자	노동호 (Dongho Roh)
문서 성격	설계·구현·운영 전반 총괄 기획안

0. 문서 요약 (Executive Summary)

본 프로젝트는 태양광 자동차가 3,038km의 호주 종단 경로를 **27일 제한시간**과 **9개 컨트롤스탑¹(CS) 마감시각**, 구간 **평균속도 60km/h 하한**, **법정 속도제한**이라는 실제 대회 규칙 아래에서 완주할 수 있도록, 매 지점의 SOC²·일사량·경사·풍향·에너지 예산을 종합해 **구간별 권장 주행속도**를 산출하는 예측·제어 시뮬레이터다. 액추에이터를 직접 제어하지 않고 드라이버에게 권장속도를 제시하는 어드바이저리³(advisory) 시스템이다.

현재 규칙 기반(Rule-based) MPC⁴와 Optuna⁵ 파라미터 최적화, 웹 시뮬레이터, 팀 공용 탐색 서버까지 동작하는 상태이며, 다음 단계는 코스팅·제동 등 물리 building block을 완성한 뒤 규칙 항들을 명시적 비용함수 기반 MPC로 재구성하는 것이다.

핵심 성과 요약

구분	내용
제어 로직	SOC·경사·일사량·에너지예산·풍향 5개 보정항 + 시간예산 블렌딩(LV1~LV8), 전 항목 연속(ramp) 방식
최적화	Optuna TPE로 12개 파라미터 탐색. 고정 5개 날씨 시나리오 평균(common random numbers)으로 로버스트 평가
웹 서비스	Streamlit Cloud 시뮬레이터 + Supabase 인증/기록 + 오라클 서버 Optuna 런치(팀 공용)
성능 개선	프로파일링 기반 병목 제거로 탐색 1회 214초 → 51초 (4.2배)
검증	리팩터링 전후 A/B 비교로 도달거리 차이 0.1% 확인, 멀티유저 격리 버그 2건 발견·수정

문서 구성

장	내용
1장	프로젝트 개요 — 배경, 목표, 산출물, 대회 규칙 제약
2장	문제 정의 — 최적화 문제로서의 정식화
3장	시스템 아키텍처 — 2앱 구조, 계층, 설계 원칙
4장	물리 모델 — 주행저항·발전·배터리·모터
5장	MPC 제어 로직 — LV1~LV8 상세와 현재 한계
6장	파라미터 최적화 — 목적함수, 탐색공간, 로버스트 설계
7장	웹 서비스 — Streamlit 앱, Supabase, Optuna 런치
8장	데이터 파이프라인 — 경로·기상·규정 데이터
9장	검증 전략 및 이력
10장	개발 현황 및 로드맵
11장	리스크 및 알려진 이슈

1) 컨트롤스탑: 대회가 지정한 의무 정차 지점. 정해진 시각 안에 도착해야 하고 일정 시간 머물러야 한다

2) SOC: State of Charge. 배터리 잔량을 0~1(또는 %)로 나타낸 값. 1이면 만충, 0이면 방전

3) 어드바이저리: 차량을 직접 조작하지 않고 '이 속도로 가세요'라고 사람에게 권하기만 하는 방식

4) MPC: Model Predictive Control(모델 예측 제어). 앞으로 벌어질 일을 짧게 내다보고 지금 무엇을 할지 정한 뒤, 한 걸음 가서 다시 계산하기를 반복하는 제어 방식

5) Optuna: 여러 설정값 조합을 자동으로 시도해 가장 좋은 조합을 찾아주는 파이썬 최적화 라이브러리

1. 프로젝트 개요

1.1 배경

World Solar Challenge(WSC)는 호주 Darwin에서 Adelaide까지 3,038km를 태양광 에너지만으로 종단하는 대회다. 차량에 탑재할 수 있는 배터리 용량과 태양광 패널 면적이 규정으로 제한되므로, **순간 최고속도보다 에너지 수지를 어떻게 배분하느냐가 완주와 순위를 결정한다.**

빠르게 달리면 공기저항이 속도의 제곱(소비전력은 세제곱)으로 증가해 배터리가 조기에 고갈되고, 느리게 달리면 27일 제한시간과 각 컨트롤스탑의 마감시각을 넘겨 실격된다. 즉 이 문제는 **에너지 제약과 시간 제약 사이의 최적 균형점을 경로 전 구간에 걸쳐 찾는 문제다.**

1.2 목표

- **1차 목표** — SOC·일사량·경사·풍향·에너지 예산을 입력으로 구간별 권장 최적 속도를 산출하는 MPC 기반 의사결정 시스템 구축 (액추에이터 직접 제어가 아닌 드라이버 어드바이저리 출력)
- **2차 목표** — 규칙 기반 제어를 명시적 비용함수와 제약조건⁶을 푸는 실제 MPC로 전환
- **3차 목표** — 발전량·소비전력 AI 예측 모델로 MPC 입력을 고도화하고, 강화학습⁷ 기반 전략과 비교
- **최종 목표** — 실차 계측 데이터로 시뮬레이션 파라미터를 교체해 실전 운영 도구로 전환

1.3 산출물

산출물	형태	상태
시뮬레이션 엔진	Python 모듈 (run_simulation)	완료
MPC 속도 플래너	Python 모듈 (mpc_speed)	완료 (규칙 기반)
파라미터 최적화	Optuna 스크립트 + 웹 런처	완료 (실기동 검증 전)
웹 시뮬레이터	Streamlit 앱 (공개 배포)	완료, v1.0.8
팀 공용 탐색 서버	FastAPI + systemd (오라클)	구현 완료, 검증 전
기술 문서	README / progress / debug_logs	지속 갱신

1.4 대회 규칙 제약 (시뮬레이션에 반영된 항목)

제약	값 / 규칙	시뮬레이션 반영 방식
총 제한기간	27일 (2027-08-23 출발 기준)	일자 초과 시 즉시 실격 처리
일일 주행시간	08:00 ~ 17:00	야간 진입 시 다음날 08:00으로 시간 점프
컨트롤스탑	9개소, 각 오픈/마감 시각 지정	마감 초과 시 실격, 정차시간 SOC 연동 30~60분
구간 평균속도	직전 CS~현재 CS 60 km/h 이상	미달 시 실격 처리
법정 속도제한	구간별 계단형 (Route Notes 기준)	제어 출력 최종 클립
신호등/보행자	실제 위치 데이터 보유	평균 대기시간 가산 (단위 버그로 현재 미발동)

6) 제약조건: 반드시 지켜야 하는 조건. 어기면 해가 될 수 없다

7) 강화학습: 시행착오를 반복하며 보상을 많이 받는 행동을 스스로 익히는 기계학습 방식

컨트롤스탑 위치·시각은 2025년 실제 대회 데이터를 사용하며, 2027년 확정 시 Configs/Vehicle_Params.py의 RaceConfig만 교체하면 된다.

2. 문제 정의

2.1 최적화 문제로서의 정식화

경로를 약 20m 간격의 이산 지점 $i = 1 \dots N$ 으로 나누고, 각 지점의 주행속도 v_i 를 결정변수로 두면 문제는 다음과 같이 쓸 수 있다.

목적: maximize $f(v) =$ 완주 시 평균속도 (미완주 시 도달률 기반 페널티)

제약: $SOC(i) \geq soc_hard_stop$ (배터리 고갈 방지)
 $T(CS_k) \leq CS_close_hour[k]$ (각 컨트롤스탑 마감시각)
 $avg_speed(leg_k) \geq 60 \text{ km/h}$ (구간 평균속도 하한)
 $v_i \leq speed_limit(i)$ (법정 속도제한)
 $v_i \leq v_max_derated(SOC)$ (전압 강하에 따른 모터 상한)
 $T_total \leq 27 \text{ days}$ (총 제한기간)

2.2 문제의 난점

- **강한 상태 의존성** — 지금의 속도 결정이 SOC를 바꾸고, 그 SOC가 이후 전 구간의 가용 선택지를 바꾼다. 각 스텝을 독립적으로 풀 수 없다.
- **미래 불확실성** — 일사량과 풍속은 예보값이며 실제와 다르다. 특정 날씨에만 잘 맞는 해는 실전에서 위험하다.
- **비선형·비대칭 페널티**⁸ — 공기저항은 v^2 , 소비전력은 v^3 , 배터리 손실은 I^2R 로 증가한다. 반면 시간 제약 위반은 즉시 실격이라 연속적인 트레이드오프가 아니다.
- **이산 이벤트 혼재** — 컨트롤스탑 정차, 야간 정지, 신호등 대기 같은 이산 이벤트가 연속적인 주행 물리와 섞여 있다.

2.3 채택한 접근

전 구간을 한 번에 푸는 대규모 최적화 대신 **2단계 구조**를 채택했다.

단계	역할	수행 주체
온라인 (매 스텝)	현재 상태를 관측하고 규칙 기반으로 즉시 권장속도 산출. receding horizon 방식으로 매 스텝 재계산	mpc_speed()
오프라인 (사전)	규칙의 가중치·임계값 14개를 전 구간 시뮬레이션 결과로 평가해 베이지안 최적화(TPE)로 튜닝	Optuna

설계 의도 — 매 스텝 수치 최적화를 푸는 대신 '잘 튜닝된 규칙'으로 근사한다. 계산량이 실시간 수준으로 작고 동작을 사람이 해석할 수 있다는 장점이 있으나, 제약을 최적화 내부에서 다루지 못하고 사후 클리핑⁹에 의존하는 한계가 있다 (5.4절 참고).

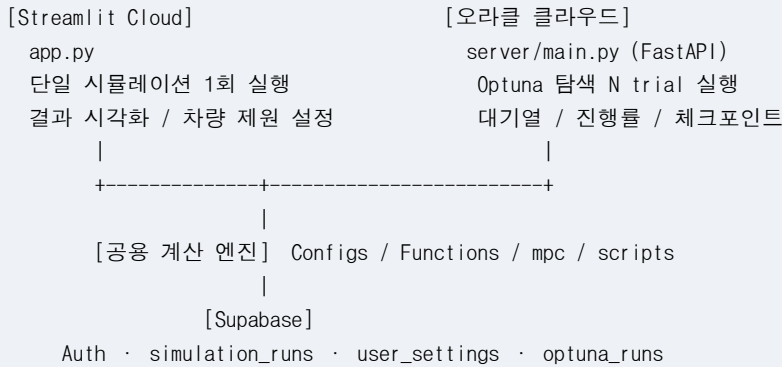
8) 페널티: 규칙을 어긴 해에 점수를 깎아 최적화가 그쪽을 피하게 만드는 장치

9) 클리핑: 계산 결과가 허용 범위를 벗어나면 범위 끝값으로 잘라내는 것

3. 시스템 아키텍처

3.1 전체 구성

시스템은 **공용 계산 엔진** 하나를 **두 개의 앱**이 공유하는 구조다. 두 앱은 같은 Supabase¹⁰ 프로젝트를 백엔드로 사용해 계정과 결과가 이어진다.



3.2 모듈 계층

계층	파일	역할
설정	Configs/Vehicle_Params.py	차량 제원 dataclass 8종 + build_default_cfg() (세션별 독립 인스턴스 생성)
물리 엔진	Functions/Vehicle_Function.py	주행저항·SOC·발전량 계산, run_simulation() 메인 루프, 헬퍼 6종
제어기	mpc/mpc_controller.py	mpc_speed() — LV1~LV8 규칙 기반 속도 결정
최적화	scripts/main.py	build_objective() — 탐색 공간 정의(단일 소스), run_cli()
웹 앱	app.py	Streamlit UI, 로그인, 결과 저장/조회, 차량 제원 편집
탐색 서버	server/	FastAPI 런처 + 별도 프로세스 탐색 실행
데이터 수집	Environment/Open_Meteo_API.py	과거 기상 데이터 수집

3.3 핵심 설계 원칙

원칙 1 — 설정은 전역이 아니라 인자로 전달한다

모든 차량 제원과 시뮬레이션 파라미터는 모듈 전역 싱글턴¹¹이 아니라 `cfg → const dict`로 함수에 전달된다. Streamlit¹² Cloud는 여러 사용자가 **서버 프로세스 하나를 공유**하므로, 전역 객체를 수정하면 한 사용자의 설정 변경이 그 순간 접속 중인 모든 사용자의 시뮬레이션 결과를 바꾼다.

이 버그는 실제로 두 번 발생했다. 1차는 차량 제원 전체(physics/solar/cell/pack/power/drive/race)가 전역이던 문제로 `build_default_cfg()` 도입으로 해결했고(progress/28), 2차는 `mpc_controller.py`가 `simpara`만 전역 참조로 남아 사용자의 설정 변경이 조용히 무시되던 문제로 `const` 전달로 해결했다(progress/36). **`run_simulation()/mpc_speed()`를 직접 호출하는 코드를 새로 작성할 때는 최신 시그니처를 반드시 확인해야 한다.**

10) Supabase: 회원가입·로그인과 데이터베이스를 대신 제공해 주는 클라우드 서비스

11) 싱글턴: 프로그램 전체에서 딱 하나만 존재하는 객체

12) Streamlit: 파이썬 코드만으로 웹 화면을 만들 수 있게 해주는 도구

원칙 2 — 탐색 공간은 한 곳에서만 정의한다

Optuna 탐색 공간은 `scripts/main.py`의 `build_objective()`에만 존재하고, 웹 런처는 이를 import해서 재사용한다. CLI로 돌리든 웹으로 돌리든 파라미터 범위가 같아지지 않는다.

원칙 3 — 원시 데이터 조회와 제어 판단을 분리한다

속도제한, 신호등 같은 원시 데이터 조회는 `Vehicle_Function.py`에서 수행해 `step dict`에 실어 보내고, `mpc_controller.py`는 그 값을 제어에 반영하기만 한다. 제어기가 데이터 소스를 직접 알지 않게 해 교체 가능성을 확보한다.

4. 물리 모델

4.1 차량 제원

항목	값	항목	값
차량 질량	250 kg	구름저항계수 Crr	0.001
항력계수 Cd	0.081 (80km/h 기준)	전면 투영면적	1.0 m ²
구동계 효율	0.99	공기밀도	1.225 kg/m ³
태양광 패널	6.0 m ² , 효율 27%	회생제동 효율	0.60
배터리 셀	Molicel P60B 6000mAh	셀 내부저항	0.0128 Ω
HV 팩	40S3P, 약 2,592 Wh	LV 팩	5S2P, 약 216 Wh
모터 정격	1,800 W / 150 V DC	정격 토크	16.2 Nm
모터 효율	0.97 (인버터 0.95)	휠 반경	0.275 m
이론 최대속도	약 155 km/h	주행 중 LV 소비	50 W

4.2 주행 저항

매 스텝 차량에 작용하는 저항력을 합산해 필요 구동력과 소비전력을 계산한다.

```

F_aero = 0.5 · ρ · Cd · A_f · v_rel2      (v_rel: 대기 상대속도, 풍향 반영)
F_roll = Crr · m · g · cos(θ)
F_slope = m · g · sin(θ)
F_acc = m · a                               (a = (v - v_prev)/dt, 실측 기반 동적 계산)

P_drive = (F_aero + F_roll + F_slope + F_acc) · v / (η_drive · η_motor · η_inv)

```

- 풍향은 차량 헤딩과의 상대각을 계산해 정면풍 성분만 유효 속도에 반영한다.
- 가속도 a 는 고정 상수가 아니라 직전 스텝 대비 실측값으로 계산한다. $dt = 0$ 인 경로 중복점에서 0으로 나누는 문제가 있어 $dt > 0$ 가드를 두었다.
- 경사각 θ 는 원본 20m 해상도 slope를 그대로 쓰지 않는다. 고도 데이터 노이즈가 과대증폭되기 때문에 500m 이동평균¹³⁾으로 스무딩한 뒤 오르막/내리막을 판정한다.

13) 이동평균: 들쭉날쭉한 값을 일정 구간씩 평균 내어 매끄럽게 만드는 방법

4.3 발전 모델

$$P_{\text{gen}} = \text{일사량}[\text{W/m}^2] \cdot A_{\text{solar}} \cdot \eta_{\text{solar}}$$

컨트롤스탑 정차 중에는 패널 각도를 태양에 맞출 수 있으므로 보정계수 $cs_chg_eff = 0.8$ 을 적용한 별도 경로로 충전을 계산한다.

4.4 배터리 모델

SOC-OCV¹⁴ 룩업 테이블(12점 선형보간)로 개방전압을 구하고, 내부저항¹⁵에 의한 전압 강하와 I^2R 손실을 반영한다.

$$\begin{aligned} V_{\text{ocv}} &= \text{interp}(\text{SOC}, \text{ocv_soc}, \text{ocv_V}) \cdot HV_S \\ R_{\text{eq}} &= R_{\text{cell}} \cdot HV_S / HV_P \\ I &= (V_{\text{ocv}} - \sqrt{V_{\text{ocv}}^2 - 4 \cdot R_{\text{eq}} \cdot P_{\text{batt}}}) / (2 \cdot R_{\text{eq}}) \\ \text{SOC}(t+1) &= \text{SOC}(t) - I \cdot dt / HV_Capa \end{aligned}$$

$\sqrt{\quad}$ 안의 값이 음수가 되면(요구 전력이 배터리 최대 출력을 초과) NaN이 발생해 이후 계산 전체가 오염된다. 실제로 $dt=0$ 버그가 이 경로를 통해 시뮬레이션 크래시로 이어진 사례가 있었다.

4.5 모터 디레이팅

SOC가 낮아지면 단자전압이 떨어져 모터가 낼 수 있는 최대 회전수가 제한된다. 매 스텝 현재 단자전압으로부터 물리적 최대속도를 계산해 제어 출력의 상한으로 사용한다.

$$\begin{aligned} \omega_{\text{max}} &= V_{\text{terminal}} / (K_v \cdot \sqrt{6}) \quad (\text{SVPWM 기준}) \\ v_{\text{max_derated}} &= \omega_{\text{max}} \cdot r_{\text{wheel}} \cdot 3.6 \end{aligned}$$

5. MPC 제어 로직

5.1 시뮬레이션 1스텝 처리 순서

경로/기상 조회 → lookahead(향후 발전량·경사) → 필요 페이스(전체완주·다음CS)
 → 모터 디레이팅 → 법정 속도제한 조회
 → $\text{mpc_speed}(\text{step}, \text{params}, \text{const})$ ← 권장속도 결정
 → dt 계산 → 야간 경계 검사 → 컨트롤스탑 정차 처리
 → 종료조건 검사(SOC / CS마감 / 구간평균속도 / 27일)
 → 주행저항·발전량으로 SOC 갱신

루프 시작부에서 먼저 달력을 확인해 현재 시각이 주행 가능 시간(08:00~17:00)이 아니면 속도·에너지 계산을 전부 건너뛰고 다음날 08:00으로 점프한다. 또한 이번 스텝의 소요시간이 당일 17:00까지 남은 시간을 넘으면 그 스텝을 계산하지 않고 시간만 17:00으로 보낸 뒤 다음 루프에서 야간 리셋한다. 불필요한 계산을 줄이기 위한 선처리다.

14) OCV: Open Circuit Voltage(개방전압). 전류가 흐르지 않을 때의 배터리 전압으로, 잔량을 추정하는 기준이 된다

15) 내부저항: 배터리 안에서 전류를 흐르지 못하게 막는 저항. 전류가 클수록 손실과 발열이 커진다

5.2 속도 결정 단계 (LV1~LV8)

단계	기준	조절 방식
LV1	SOC (soc_ramp_low ~ soc_ramp_high)	$v_{min} \sim v_{soc_high}$ 선형보간으로 기저속도 결정
LV2	경사 look-ahead (현재~2km 구간평균) + 직전 가속도	slope_k 비례 감속. 오르막 진입 시 momentum_gain-a 만큼 페널티 완화
LV3	일사량 비율	radi_para 비례 가감속. radi_risk-표준편차로 불확실성 할인
LV4	에너지 예산 (다중지점 가중평균 발전량 기반)	잔여 구간 필요 SOC 대비 예상 SOC 부족분에 energy_v 비례 감속
LV5	정면풍 성분	winddir_para 비례, 대칭 클리핑
LV8	시간 예산 (전체완주 페이스 + 다음CS 페이스)	SOC 여유에 비례해 목표 페이스로 3항 가중평균 블렌딩

LV6은 설계 당시 결번이고, 구 LV7(미래 일사량 추세)은 LV4에 병합됐다. 파라미터 이름은 LV 번호 접두어를 떼고 역할 기반 이름(v_{soc_high} , slope_k 등)을 쓴다 — 로직 순서가 바뀔 때마다 번호를 다시 매겨야 하는 문제 때문에 폐기했다.

5.3 클리핑 순서와 댐퍼

보정이 모두 누적된 뒤 다음 순서로 상하한을 적용한다. **순서가 중요하다.**

1. $v = \max(v, v_{min})$ 누적 보정으로 v 가 0 이하가 되는 것 방지
2. $SOC \leq soc_cutoff$ 이면 $v = v_{min}$ 하드 안전장치 (계단형)
3. LV8 시간예산 블렌딩
4. $v = \min(v, v_{max_derated}, drive.v_{max})$ 물리적 상한
5. $v = \min(v, speed_limit)$ 법정 속도제한 ← 반드시 v_{min} 클램프보다 뒤
6. 댐퍼: $v = v_{prev} + \Delta_{max} \cdot \tanh(\alpha \cdot (v - v_{prev}) / \Delta_{max})$

5번이 1번보다 앞에 오면, v_{min} 강제 로직(예: 60km/h)이 그보다 낮은 법정 속도제한(예: CS 진입 25km/h)을 덮어써서 규정 위반 주행이 된다. 실제로 이 순서 문제를 검증 과정에서 확인하고 고정했다.

댐퍼¹⁶는 EMA+tanh 형태로, 빠른 오실레이션은 감쇠시키되 느린 추세는 그대로 따라가고 한 번의 큰 이상치는 차량의 물리적 가감속 한계에서 포화되도록 설계했다. 이전에는 단순 rate limiting을 썼으나 경계에서 속도가 고착되는 문제가 있었다.

5.4 현재 모델의 한계 (실제 MPC 정의 대비)

구분	항목
없는 것	명시적 예측 모델 기반 수치 최적화(매 스텝 비용함수를 실제로 푼), 명시적 objective function, 제약조건을 최적화 내부에서 다루는 것 (현재는 soc_cutoff / v_{max} / speed_limit 전부 사후 클리핑)
있는 것	receding horizon(매 스텝 상태 재측정 후 재계산), 상태 피드백, 부분적 예측(avg_gen_ratio, slope_ahead)

전환 순서는 합의돼 있다. 코스팅·제동·내리막 캡 같은 물리 building block을 규칙 기반으로 먼저 완성해 물리적 타당성을 검증한 뒤, 같은 항들을 비용함수와 solver로 재구성한다. 오르막 처리는 배터리 I²R 손실(제곱 손실) 기반 비용항으로, LV4의 균등 배분 가정은 지형 기반 1회성 예산 배분으로 재설계할 예정이다.

16) 댐퍼: 값이 급격히 출렁이는 것을 눌러 부드럽게 만드는 장치

6. 파라미터 최적화

6.1 목적함수

```
for each of K=5 fixed weather scenarios:
    df, reason = run_simulation(params, ...)
    if 완주:      score = 평균속도 [km/h]
    else:        score = 도달률 - 2      (범위 [-2, -1])

objective = mean(scores)
```

완주 여부가 1차 기준이고 그 안에서 평균속도를 겨루는 구조다. 미완주 해에 -2 오프셋을 주어 어떤 완주 해보다도 항상 낮게 만들되, 도달률을 더해 '얼마나 아깝게 실패했는지'의 기울기를 남겨 탐색이 방향을 잡을 수 있게 했다.

6.2 탐색 공간 (12개 파라미터)

파라미터	범위	의미
v_min	20 ~ 60 (int)	속도 하한 [km/h]
v_soc_high	60 ~ 100 (int)	SOC 충분 시 목표속도 [km/h]
soc_ramp_high	0.5 ~ 1.0	LV1 램프 상단 SOC 임계값
soc_ramp_low	0.0 ~ 0.5	LV1 램프 하단 SOC 임계값
slope_k	0 ~ 150 (int)	경사 보정 계수
radi_para	0 ~ 40 (int)	일사량 보정 계수
radi_risk	0.0 ~ 1.0	일사량 불확실성 할인 계수
energy_v	0 ~ 20	에너지 예산 보정 계수
winddir_para	0 ~ 20 (int)	풍향 보정 계수
margin_total	0.1 ~ 0.6	전체 완주 페이스 블렌딩 여유
margin_next_cs	0.1 ~ 0.6	다음 CS 페이스 블렌딩 여유
soc_cutoff	0.00 ~ 0.20	하드 세이프티 SOC 하한

momentum_gain과 alpha는 현재 고정값이며, 본 탐색 재실행 시 탐색 공간 포함 여부를 결정한다. 파라미터 이름은 세 지점(기본값 dict, mpc_speed 본문, suggest 호출부)이 모두 일치해야 하며, 하나라도 다르면 조용히 기본값으로 fallback되므로 주의가 필요하다.

6.3 로버스트 설계 — 날씨 처리

고정 시나리오 (Common Random Numbers)

매 trial마다 새 날씨를 뽑으면 '운 좋은 날씨를 만난 trial'이 이겨버려 파라미터의 실력을 비교할 수 없다. 고정 시드 5개로 미리 만든 동일한 날씨 세트에서 모든 trial을 평가한다.

CS 구간 단위 상관 섭동

예보값에 표준편차¹⁷를 곱해 흔들 때, 지점마다 독립적인 난수를 쓰면 바로 옆 지점의 일사량이 뜬금없이 튀는 비현실적 노이즈가 된다. 실제 날씨는 전선 단위로 뭉쳐 움직이므로, **같은 CS 구간(leg) 안에서는 같은 z값을 공유**하도록 바꿨다. 한 leg 안에서는 '이번엔 대체로 맑은/흐린 구간'처럼 일관되게 흔들리고, leg가 바뀌면 새로 뽑힌다.

6.4 샘플러 선택

TPE¹⁸(Tree-structured Parzen Estimator)에 `multivariate=True`, `group=True` 옵션을 적용했다. 목적함수¹⁹가 K개 평균으로 평활화되었더라도, 완주/미완주 경계에서 목적함수가 불연속적으로 튀는 특성 때문에 CMA-ES보다 TPE가 안정적이라고 판단했다.

6.5 병렬화 전략

Optuna의 `n_jobs`는 GIL²⁰ 때문에 CPU-bound 순수 Python 루프에서 실효성이 없다. 진짜 병렬을 얻으려면 **OS 프로세스를 분리**해 같은 storage에 붙여야 하며, 이때 SQLite²¹는 WAL 모드²²로 열어야 동시 접근이 가능하다. 웹 런처가 이 방식으로 구현되어 있다.

7. 웹 서비스 아키텍처

7.1 Streamlit 앱 (app.py)

기능	설명
경로 애니메이션	호주 실루엣 배경 위에 주행 진행을 실시간 표시. 배경/전체경로는 최초 1회만 그리고 이후 마커·주행경로만 갱신하는 커스텀 JS 컴포넌트
시뮬레이션 실행	로그인한 사용자만 실행 가능. 진행률 콜백으로 실시간 상태 표시
결과 분석	종료 사유(SOC/CS마감/구간속도/27일/완주)별로 분기한 규칙 기반 진단 메시지
차량 제원 설정	9개 탭(물리·태양광·셀·팩·전력·구동계·레이스·시뮬레이션·OCV)에서 직접 편집. 세션별로 격리되며 계정에 저장/불러오기 가능
기록 관리	결과를 계정에 저장하고 사이드바에서 조회. 각 기록에서 그때 사용한 차량 제원 복원
Optuna 결과 연동	서버에서 돌린 탐색 결과를 조회하고 '이 파라미터로 시뮬레이션하기'로 즉시 적용
부가 기능	자동 로그인(refresh token), 릴리즈 노트, 첫 로그인 튜토리얼, GitHub 원본 코드 뷰어

17) 표준편차: 값이 평균에서 얼마나 흩어져 있는지 나타내는 수치. 클수록 불확실하다

18) TPE: Optuna가 쓰는 탐색 방법. 지금까지의 시도 결과로 '좋은 값이 있을 법한 구간'을 추려 다음 후보를 고른다

19) 목적함수: '무엇을 잘한 것으로 볼지'를 숫자 하나로 정의한 식. 최적화는 이 값을 키우거나 줄인다

20) GIL: 파이썬이 한 번에 한 흐름만 실행하도록 거는 잠금장치. 이 때문에 스레드를 늘려도 계산이 빨라지지 않는다

21) SQLite: 파일 하나로 동작하는 가벼운 데이터베이스

22) WAL 모드: SQLite에서 쓰기와 읽기가 서로 막지 않게 해주는 저장 방식

7.2 Supabase 스키마

테이블	용도	주요 컬럼
auth.users	인증 (Supabase 기본)	user_metadata에 nickname, tutorial_seen 저장
simulation_runs	시뮬레이션 실행 기록	params, completion_ratio, avg_speed_kmh, final_soc, final_dist_m, vehicle_cfg
user_settings	사용자별 마지막 차량 제원	user_id(PK), vehicle_cfg(jsonb)
optuna_runs	Optuna 탐색 진행/결과	study_name(고유), n_trials_completed/target, best_value, best_params, status

모든 테이블에 RLS²³(Row Level Security)를 적용해 auth.uid() = user_id 조건으로 본인 행만 접근 가능하다. 클라이언트에 노출되는 anon key는 RLS를 전제로 공개해도 되는 값이며, service_role key는 절대 클라이언트에서 사용하지 않는다.

보완 필요 — optuna_runs 테이블의 생성 SQL 원문이 저장소에 남아있지 않아, 현재는 코드에서 컬럼을 역추적해야 한다. Supabase 재구축 상황에 대비해 SETUP.md에 정리가 필요하다.

7.3 Optuna 웹 런처 (server/)

Streamlit Cloud 무료 티어로는 수십 분이 걸리는 탐색을 돌릴 수 없고, 돌린다 해도 서버 프로세스를 공유하는 다른 사용자에게 영향을 준다. 따라서 탐색만 오라클 클라우드의 별도 서버로 분리했다.

로그인 (Supabase 토큰을 서버가 Supabase에 검증 요청)

- 대기열 등록 (동시 실행 WSC_MAX_CONCURRENT=1, 최대 trial 100)
- subprocess로 study_runner.py 실행
- 매 trial마다 progress.json 갱신 → 진행률 · 예상 잔여시간 표시
- 20 trial마다 optuna_runs에 체크포인트 upsert
- 완료 시 best_params로 최종 시뮬레이션 1회 + 결과 파일 저장

설계 판단

- **왜 subprocess²⁴인가** — study.optimize()가 블로킹²⁵ 호출이라 FastAPI²⁶ 프로세스 안에서 돌리면 다른 사용자의 요청까지 막힌다. 스레드로는 GIL 때문에 실제 병렬이 되지 않는다.
- **왜 자체 사용자 DB를 안 만들었나** — 이미 있는 Supabase Auth를 재사용해 계정 체계를 하나로 유지했다. 서버는 받은 토큰²⁷을 Supabase에 물어봐 검증만 한다.
- **디스크 로테이션** — 무료 인스턴스의 작은 디스크가 차면, 지금까지의 best_params를 enqueue_trial로 이어받은 새 study로 교체해 탐색을 계속한다. 결과는 이미 Supabase 체크포인트²⁸에 있어 안전하다.
- **진행률 파일 분리** — study sqlite를 직접 세지 않고 별도 progress.json을 읽는다. 로테이션이 일어나도 경로가 바뀌지 않아 진행률 표시가 끊기지 않는다.

23) RLS: Row Level Security. 데이터베이스가 '이 사람은 자기 줄만 볼 수 있다'를 강제하는 보안 기능

24) subprocess: 프로그램이 별도의 독립된 프로그램을 하나 더 띄워 실행하는 것

25) 블로킹: 그 작업이 끝날 때까지 다른 일을 못 하고 멈춰 있는 상태

26) FastAPI: 파이썬으로 웹 서버(API)를 만드는 도구

27) 토큰: 로그인했음을 증명하는 임시 문자열. 비밀번호 대신 주고받는다

28) 체크포인트: 중간 결과를 저장해 두어, 중간에 멈춰도 거기서부터 살릴 수 있게 하는 것

8. 데이터 파이프라인

8.1 데이터 소스

데이터	출처	규모 / 형태
경로	2027 BWSC TRACK.csv	Darwin~Adelaide 3,038km, 약 20m 간격 GPS + 고도
기상	Open-Meteo 과거 데이터 API	305좌표 × 8일 × 12시간 = 29,280행
속도제한	공식 Route Notes PDF에서 추출	구간별 계단형 제한속도
신호등	공식 Route Notes PDF에서 추출	위치 + 종류(차량/보행자)
컨트롤스탑	2025 대회 실제 데이터	9개소 위치·오픈/마감 시각

8.2 전처리

- **경사 스무딩** — 20m 해상도 원본 slope는 고도 노이즈가 과대증폭되므로 500m 이동평균 후 $C_{rr} \cdot \cos(\theta) + \sin(\theta)$ 부호로 오르막/내리막을 판정한다.
- **기상 격자 매핑** — 경로의 각 지점을 가장 가까운 기상 관측 지점에 사전 매핑해 런타임 조회를 $O(1)$ 로 만든다.
- **메모리 절감** — 실제 사용하는 5개 컬럼만 남기고 dict로 변환한다. $K=5$ 개 날씨 사본을 동시에 들고 있어야 해서 1GB 메모리 서버에서는 이 최적화가 필수다.
- **numpy 변환** — 거리 배열을 pandas Index가 아닌 numpy 배열로 유지한다 (9.2절 참고).

8.3 지형 세그먼트

연속된 내리막 구간의 경계를 `np.diff/np.where`로 찾아 [시작거리, 끝거리, 대표경사각] 배열로 변환하는 1회성 전처리가 구현되어 있다. 이 배열은 코스팅 진입 속도 상한 계산에 쓰일 예정이나, 이를 소비하는 `compute_downhill_cap()`이 아직 구현되지 않아 현재는 계산만 하고 사용되지 않는다.

9. 검증 전략 및 이력

9.1 검증 원칙

- 동작을 바꾸지 않아야 하는 리팩터링²⁹은 **변경 전후 A/B³⁰ 비교**로 결과 동일성을 확인한다.
- 성능 문제는 추측하지 않고 **cProfile로 실측**한 뒤 병목을 특정한다.
- 멀티유저 관련 수정은 **격리 테스트**(두 설정이 서로 영향을 주지 않으면서 각자 결과에는 반영되는지)로 검증한다.
- 코드를 작성한 세션이 실행 환경이 없어 검증하지 못한 경우, 다음 세션이 **대신 검증**하고 결과를 문서에 남긴다.

29) 리팩터링: 동작은 그대로 두고 코드 구조만 정리하는 작업

30) A/B: 바꾸기 전(A)과 바꾼 후(B)를 같은 조건으로 돌려 결과를 비교하는 검증 방법

9.2 주요 검증 사례

사례 1 — 함수 분리 리팩터링 A/B 검증

200줄이 넘는 `run_simulation()`을 6개 헬퍼 함수로 분리(약 160줄로 축소)한 뒤, 동일 입력에 대한 도달거리 차이가 0.1%임을 확인해 동작 보존을 검증했다.

사례 2 — 성능 병목 특정 (4.2배 개선)

탐색 시간이 갑자기 8분대로 늘어난 원인을 두 가지 가설(크래시 조기종료, 조기 실격)로 추정했으나 모두 반증됐다. cProfile로 직접 측정한 결과 `dist_vals`가 numpy 배열이 아닌 pandas Index여서 lookahead 샘플링 루프가 매번 pandas의 무거운 내부 경로를 타고 있었고, 이것이 전체 실행시간의 74%를 차지했다. `.to_numpy()` 한 줄로 214초 → 51초로 개선했고 결과는 동일함을 확인했다.

교훈 — 성능 문제는 가설을 세우는 것보다 프로파일³¹러를 먼저 돌리는 것이 빠르다.

사례 3 — 멀티유저 격리 버그 2건

Streamlit Cloud가 서버 프로세스를 공유한다는 특성 때문에 전역 상태가 사용자 간에 새는 버그가 두 번 발생했다. 두 경우 모두 '전역을 잠시 바꿨다가 되돌리는' 방식을 검토했으나, Streamlit이 세션마다 별도 스레드로 실행되고 시뮬레이션이 수십 초~수 분 도는 동안 바이트코드 수준 인터리빙이 일어날 수 있어 근본적으로 안전하지 않다고 판단하고 인자 전달 구조로 재설계했다.

사례 4 — 야간 경계 선처리 검증

루프 순서 변경 후 전 구간 시뮬레이션이 무한루프 없이 정상 종료함을 확인했다. 도달거리는 2,833,560m → 2,833,500m(-60m, 0.002%)로, partial segment 분할을 아직 하지 않아 17시 직전 잔여 거리를 보수적으로 버린다는 설계 설명과 일치했다. 경계 조건(등호 포함 비교, 잔여시간 0인 경우)도 검토해 이중 처리와 무한루프가 없음을 확인했다.

9.3 발견된 주요 버그 이력

버그	영향	상태
야간 데이터 처리 (HR==17에 날짜 미전환)	도달거리 993km(35%) 차이	수정
dt=0 divide-by-zero → NaN 전파	시뮬레이션 크래시	수정 (가드 추가)
차량 제원 전역상태 누수	한 사용자 설정이 전체에 반영	수정 (cfg 격리)
mpc_controller simpara 전역 참조	사용자 설정이 조용히 무시됨	수정 (const 전달)
dist_vals pandas Index	실행시간 4.2배	수정
법정 속도제한 클립 순서	속도제한 무시 가능	수정 (순서 고정)
light_arrive() 단위 불일치 (m vs km)	신호등 지연 미발동	미수정 (보류 결정)

31) 프로파일: 프로그램의 어느 부분에서 시간이 얼마나 쓰이는지 실제로 측정하는 것

10. 개발 현황 및 로드맵

10.1 단계별 현황

#	단계	상태
1	경로·환경 데이터 수집 / 차량 물리 모델	완료
2	Rule-based MPC (LV1~LV5 ramp + LV8 페이스 블렌딩)	완료
3	Streamlit 시뮬레이터 UI	완료
4	웹 배포 (Streamlit Cloud + Supabase + Optuna 런처)	완료 (런처 실기동 검증 전)
5	MPC 물리 building block (코스팅/제동/내리막 캡)	진행 중
6	비용함수 기반 MPC로 전환	설계 논의 완료
7	AI 예측 모델 (발전량 / 소비전력)	미착수
8	강화학습 실험	미착수
9	실측 데이터 교체	미착수

10.2 단기 로드맵

1순위 — Optuna 웹 런처 실제 동작 검증

서버 코드가 인터넷이 없는 환경에서 작성되어 **한 번도 실제로 기동해본 적이 없다**. 다음을 순서대로 확인해야 한다.

- 오라클 서버에서 uvicorn 기동 및 접속 확인 (포트 8000 Security List 개방 필요)
- 로그인 → 탐색 시작 → 진행률 표시 → 앱에서 결과 조회 → 파라미터 적용까지 관통 확인
- 배포본에서 자동 로그인·코드 뷰어 동작 확인
- 문제 없으면 systemd³² 등록으로 재부팅 자동 기동 구성

2순위 — MPC 물리 building block 완성

설계는 완료되어 있고 코드만 없는 상태다.

- **compute_downhill_cap(step, const)** — 내리막 세그먼트 진입 전 목표속도 상한 계산. $a_{coast} = (F_{aero} + F_{roll} + F_{slope})/m$, $v_{entry_max} = \sqrt{v_{exit_cap}^2 - 2 \cdot a_{coast} \cdot L}$
- **CS 접근 코스팅/제동** — $coast_distance = v^2 / (2 \cdot a_{coast})$ 를 트리거로 가속 해제. 내리막 등 $a_{coast} \leq 0$ 구간은 decel_brake(0.7g)로 기계 제동. 코스팅 중에는 배터리를 거치지 않고 LV 소비전력만 반영
- 완성되면 app.py에도 동일한 세그먼트 전처리 추가 필요 (현재 scripts/main.py에만 존재)
- 이후 light_arrive() 단위 버그 수정 → 본격 Optuna 재탐색

3순위 — 문서/인프라 정리

- optuna_runs 테이블 SQL을 SETUP.md에 정리 (현재 저장소에 원문 없음)
- SETUP.md에 server/ 배포 절차 링크 추가
- 서버 보안 검토 (CORS 와일드카드, anon key 하드코딩의 노출 범위)

32) systemd: 리눅스에서 프로그램을 서비스로 등록해 재부팅 후에도 자동으로 켜지게 하는 장치

10.3 장기 방향

- **비용함수 MPC 전환** — 오르막은 배터리 I²R 손실 기반 비용함수로, 에너지 예산은 균등 배분 대신 지형 기반 1회성 배분으로 재설계
- **AI 예측 모델** — 발전량·소비전력 예측 모델로 MPC 입력을 고도화
- **강화학습 비교** — 동일 환경에서 RL 정책과 MPC 전략의 성능 비교
- **실측 데이터 교체** — 실차 계측값으로 물리 파라미터를 대체해 실전 운영 도구로 전환

11. 리스크 및 알려진 이슈

11.1 기술 리스크

리스크	영향	대응
Optuna 런처 미검증	실제 배포 시 동작 실패 가능성	1순위 작업으로 지정, 배포 후 즉시 관통 검증
서버 재시작 시 진행 상태 소실	진행 중이던 탐색 유실	완료 결과는 파일+Supabase에 남으므로 치명적이지 않음. 필요 시 상태 영속화
오라클 무료 인스턴스 사양	동시 실행 1개로 제한	대기열로 처리. 사양 상향 시 설정값만 변경
신호등 지연 미발동	실제보다 낙관적인 시간 추정	단위 변환 수정 (보류 중)
partial segment 미분할	17시 직전 잔여 거리 손실	20m 해상도라 영향 미미(0.002%), 필요 시 ds_eff 분할로 정밀화

11.2 모델 신뢰성 리스크

- **실측 데이터 부재** — 현재 모든 물리 파라미터가 카탈로그 값과 이론식 기반이다. 특히 회생제동³³ 효율(0.6)과 Cd(0.081)는 실측 검증이 필요하다. 회생제동 효율 측정 절차는 코드 주석에 기록되어 있다.
- **기상 예보 오차** — 과거 데이터 기반 시뮬레이션이라 실전 예보 오차는 반영되지 않는다. K=5 시나리오 평균이 부분적으로 이를 보완한다.
- **momentum 결합 효과 미검증** — LV2의 momentum 향이 실제 SOC를 개선하는지는 A/B 검증 전이다. 물리적 타당성 검토(회생제동 손실 감안 시 '가속 후 회생'은 오히려 손해)는 완료했다.

11.3 운영 리스크

- **다중 세션 병행 작업** — 여러 컴퓨터와 여러 AI 에이전트(Claude Code / Codex)가 동시에 작업하면서 백로그 파일이 두 개로 갈라지거나, 코드만 올라오고 문서가 누락되는 일이 있었다. → 저장소의 문서 규칙(README·백로그·작업기록)으로 대응
- **커밋 누락** — 구현·테스트를 마치고 push를 잊어 배포에 반영되지 않은 사례가 있었다. → '완료 보고 전 커밋·푸시 완료' 규칙 명문화

— 문서 끝 —

33) 회생제동: 감속할 때 모터를 발전기처럼 써서 운동에너지 일부를 배터리로 되돌리는 기능